OBJECT TECHNOLOGIES™

Inheritance

www.object.co.in

What will be covered

OBJECT

- Inhertance basics
- extends keyword
- What is acquired by child
- Accessing super class members
- Constructors and Inheritance
- Example
- Method overriding
- Nested classes and Inheritance
- Final class and Inheritance

Basics

OBJECT TECHNOLOGIES

- It is one of the fundamental concepts of object-oriented programming.
- *Inheritance* is a programming construct that software developers use to establish *is-a relationships* between categories.
- Inheritance enables us to derive more-specific categories from more-generic ones. The more-specific category *is a* kind of the more-generic category.
- For example, a checking account is a kind of account in which you can make deposits and withdrawals. Similarly, a truck is a kind of vehicle used for hauling large items.
- Inheritance can descend through multiple levels, leading to ever-more-specific categories.

extends keyword

- The extends keyword specifies a parent-child relationship
- Java supports class extension via the extends keyword. When present, extends specifies a parent-child relationship between two classes.

```
class Account
{
    // member declarations
}
class SavingsAccount extends Account
{
    // inherit accessible members from Account
    // provide own member declarations
}
```

- SavingsAccount *is a* specialized Account

What is acquired by child

OBJECT

- Child classes inherit accessible fields and methods from their parent classes and other ancestors.
- They never inherit constructors, however. Instead, child classes declare their own constructors.
- Furthermore, they can declare their own fields and methods to differentiate them from their parents.
- Within a class, a field that has the same name as a field in the superclass hides the superclass's field, even if their types are different.
- An instance method in a subclass with the same signature (name, plus the number and the type of its parameters) and return type as an instance method in the superclass *overrides* the superclass's method.
- If a subclass defines a static method with the same signature as a static method in the superclass, then the method in the subclass *hides* the one in the superclass.

Constructors and Inheritance

OBJECT

- The **constructor** in the superclass is responsible for building the object of the superclass and the constructor of the subclass builds the object of subclass.
- When the subclass constructor is called during object creation, it by default invokes the default constructor of super-class.
- Hence, in inheritance the objects are constructed top-down. i.e. order of constructor calling is from super class to sub class.
- The superclass constructor can be called explicitly using the keyword super, but it should be first statement in a sub class constructor.
- The keyword super always refers to the superclass immediately above of the calling class in the hierarchy.
- The use of multiple super keywords to access an ancestor class other than the direct parent is illegal.

Example

OBJECT TECHNOLOGIES

```
public abstract class Emp extends Person
{
      private int empid;
      protected double salary;

      public Emp()
      {
            //super();
            empid=0;
            salary=0.0;
      }

      public Emp(String name,int dd,int mm,int yy,int empid,double
salary)
      {
            super(name,dd,mm,yy);
            this.empid=empid;
            this.salary=salary;
      }

      public void display()
      {
            super.display();
            System.out.println("Empid : "+empid);
```

```
public class Person
{
      private String name;
      private Date bdate;
```

Example explained

OBJECT TECHNOLOGIES

- Person class has data members as name and bdate.
- Emp class extends from Person and adds its own data members as empid and salary.
- In the full parameterized constructor of Emp(subclass), constructor of Person(superclass) is called to initialize the members of Person class by using the statement :

  super(param1,param2,param3,.....)
- In the default constructor of Emp (subclass) class, default constructor of super class is automatically called even if explicit call to super() is missing
- Other than constructor of super class, super class members are even are accessible using super keyword e.g

  super.display();

Method overriding

OBJECT TECHNOLOGIES

- Declaring a method in sub class which is already present in parent class is known as method overriding.
- Overriding is done so that a child class can give its own implementation to a method which is already provided by the parent class. In this case the method in parent class is called overridden method and the method in child class is called overriding method.
- Simply it is nothing but changing method implementation in a subclass.
- This is helpful when a class has several child classes, so if a child class needs to use the parent class method, it can use it and the other classes that want to have different implementation can use overriding feature to make changes without touching the parent class code.

```
class Parent {

    void Print()
    {
        System.out.println("parent class");
    }
}

class subclass1 extends Parent {

    void Print()
    {
        System.out.println("subclass1");
    }
}
```

Rules for Method overriding

OBJECT TECHNOLOGIES

- The argument list of overriding method (method of child class) must match the Overridden method(the method of parent class). The data types of the arguments and their sequence should exactly match.
- Access Modifier of the overriding method (method of subclass) cannot be more restrictive than the overridden method of parent class.
- private, static and final methods cannot be overridden as they are local to the class. However static methods can be re-declared in the sub class.
- Overriding method (method of child class) can throw unchecked exceptions, regardless of whether the overridden method(method of parent class) throws any exception or not. However the overriding method should not throw checked exceptions that are new or broader than the ones declared by the overridden method
- If a class is extending an abstract class or implementing an interface then it has to override all the abstract methods unless the class itself is a abstract class.

Annotation @override

OBJECT TECHNOLOGIES

- When overriding a method, you might want to use the @Override annotation that instructs the compiler that you intend to override a method in the superclass.
- Using @Override annotation while overriding a method is considered as a best practice for coding in java because of the following two advantages:

  1) If programmer makes any mistake such as wrong method name, wrong parameter types while overriding, you would get a compile time error. As by using this annotation you instruct compiler that you are overriding this method. If you don't use the annotation then the sub class method would behave as a new method (not the overriding method) in sub class.

  2) It improves the readability of the code. So if you change the signature of overridden method then all the sub classes that overrides the particular method would throw a compilation error, which would eventually help you to change the signature in the sub classes.

Nested Classes and Inheritance

OBJECT TECHNOLOGY

- Nested classes which are declared private are not inherited.
- Nested classes with the default (package) access modifier are only accessible to subclasses if the subclass is located in the same package as the superclass.
- Nested classes with the protected or public access modifier are always inherited by subclasses.

```
class MyClass {

    class MyNestedClass {

    }
}

            public class MySubclass extends MyClass {

                public static void main(String[] args) {
                    MySubclass subclass = new MySubclass();

                    MyNestedClass nested =  subclass.new MyNestedClass();
                }
            }
```

- it is possible to create an instance of the nested class MyNestedClass which is defined in the superclass (MyClass) via a reference to the subclass (MySubclass).

Final Classes and Inheritance

OBJECT TECHNOLOGIES

- A class can be declared final

```
public final class MyClass {

}
```

- A final class cannot be extended. In other words, you cannot inherit from a finalclass in Java.
- Example of final class in java library is String class

Assignments

1. Create a class Emp inheriting from Person. Add extra state like empid, salary and override method display in Emp. Call display method for Person object and Emp object.
2. In the main method create object of Emp with reference of Person. Call its display method. Check whether display method gets called from Person or Emp.
3. Create a class Customer having emailid and address. Create a subclass RegCustomer by adding data member as reg_no. Accept the type of the customer from user and accordingly accept the information from the user. Write method giveDiscount() in Customer and RegCustomer which needs input parameter as shopping price. Give no discount to Customer but give 5% discount RegCustomer and display the final price.
4. Create a class Doctor inheriting from Person class. Add extra state like degree,regno, salary in Doctor class and override method display in Doctor. Write tostring() method to display doctor and person details.
5. Create ColorPoint class extending from class Point. Add the data member as color(String) and static data member as colors(String []). In the parameterised constructor compare the color with array elements, if it is equal to any of the element, assign the value else assign "white" color value. Create the instance and verify.

Assignments-1

1. Create a class Emp inheriting from Person. Add extra state like empid, salary and override method display in Emp. Call display method for Person object and Emp object.
2. In the main method create object of Emp with reference of Person. Call its display method. Check whether display method gets called from Person or Emp
3. Override toString() method in class Person, Emp and Date. In main method call toString() method on Emp instance and display the string representation of Emp.
4. Create a class Customer having emailid and name. Create a subclass RegCustomer by adding data member as reg_no. Override toString() in Customer and RegCustomer and call toString() method for RegCustomer in main method.
5. Create ColorPoint class extending from class Point. Add the data member as color(String) and static data member as colors(String []). In the parameterised constructor compare the color with array elements, if it is equal to any of the element assign the value else assign "white" color value. Create the instance and verify.

Assignments-2

1. Create a class Emp inheriting from Person. Add extra state like empid, salary and override method display in Emp. Call display method for Person object and Emp object.

3. Override toString() method in class Person, Emp and Date. In main method call toString() method on Emp instance and display the string representation of Emp.
4. Create a class Customer having emailid and name. Create a subclass RegCustomer by adding data member as reg_no. Override toString() in Customer and RegCustomer and call toString() method for RegCustomer in main method.
5. Create ColorPoint class extending from class Point. Add the data member as color(String) and static data member as colors(String []). In the parameterised constructor compare the color with array elements, if it is equal to any of the element assign the value else assign "white" color value. Create the instance and verify.

Assignments-2

1. Create a class Emp inheriting from Person. Add extra state like empid, salary and override method display in Emp. Call display method for Person object and Emp object.
2. Create a class Customer having emailid and name. Create a subclass RegCustomer by adding data member as reg_no. Accept the type of the customer from user and accordingly accept the information from the user. Write method giveDiscount() in Customer and RegCustomer which needs input parameter as shopping price. Give no discount to Customer but give 5% discount RegCustomer and display the final price.
3. Create ColorPoint class extending from class Point. Add the data member as color(String) and static data member as colors(String []). In the parameterised constructor compare the color with array elements, if it is equal to any of the element assign the value else assign "white" color value. Create the instance and verify.

Important Questions for: Inheritance

Interview Questions

Difference between abstraction and interface

Coding Questions

There were three classes A,B and C. Class B was extending A, class C was extending B. Classes A,B,C were having default constructors which were printing "A","B" and "C" respectively.

que.1- A a=new C();

what will be printed ?

que.2 - C c=new A()

Is it valid? Explain why?